

Multi-Level Feedback Queue Scheduler on **xv6**-RISC-V

Consolidated Final Report

Muhammad Hamza Kashif - 29022 Zuhair Amirali Merchant - 29021

December 3, 2025

Abstract

This report summarizes the design, implementation, and evaluation of a Multi-Level Feedback Queue (MLFQ) scheduler for **xv6**-RISC-V. The work was carried out in three stages: initial scaffolding and observability, core scheduler integration, and starvation prevention through global priority boosting and final polishing.

Contents

1	Introduction	2
2	Background and Design Goals	2
3	Week 1: Setup and Design	2
3.1	Scope	2
3.2	New Syscall: <code>getprocinfo</code>	2
3.3	User Tool: <code>procinfo</code>	3
3.4	Process Bookkeeping Fields	3
3.5	MLFQ Parameters and Queue Structure	3
4	Week 2: Core Scheduler Implementation	3
4.1	Queue Structures and Locking	3
4.2	Enqueue and Dequeue Policy	3
4.3	Scheduler Tick Accounting	4
4.4	Scheduler Loop	4
4.5	Queue and Wakeup Policy	4
4.6	Demo Program	4
5	Week 3: Priority Boosting and Polishing	4
5.1	Global Priority Boosting	4
5.2	Boost Procedure	5
5.3	Testing Syscall: <code>boostproc()</code>	5
5.4	Scheduler Integration and Behavior	5
5.5	Observations from Demo Runs	5
6	Testing and Evaluation	5
7	Conclusion and Future Work	6

1 Introduction

The original `xv6` scheduler uses a simple round-robin policy that treats all runnable processes identically. While simple, this approach does not distinguish between CPU-bound and I/O-bound workloads and can degrade interactivity when long-running CPU tasks dominate the processor.

To address this limitation, we designed and implemented an MLFQ scheduler for `xv6-RISC-V`. The main objectives were:

- Introduce multiple priority levels with different time quanta.
- Track per-process runtime and queue metadata in the kernel.
- Prefer interactive and I/O-bound tasks while keeping CPU-bound tasks fairly scheduled.
- Prevent starvation by periodically boosting process priorities.

The project was organized into three weekly milestones:

1. Week 1: Design and observability tools.
2. Week 2: Core MLFQ scheduler integration.
3. Week 3: Priority boosting, starvation prevention, and polishing.

2 Background and Design Goals

Our design follows the standard MLFQ idea: processes start in a high-priority queue with a short time slice. If a process uses its full slice without yielding, it is treated as CPU-bound and moved to a lower-priority queue with a longer time slice. Processes that frequently block for I/O tend to remain at higher priorities and observe lower response times.

The main design goals were:

- Use a small, fixed number of queues with simple FIFO ordering.
- Keep the scheduler logic clear, with a single function that chooses the next runnable process.
- Maintain accurate per-process metrics to debug and analyze scheduling behavior.
- Add a global boost mechanism so long-running processes eventually return to the top queue.

3 Week 1: Setup and Design

3.1 Scope

Week 1 focused on preparing the kernel for MLFQ scheduling without yet changing the behavior of the existing scheduler. The work concentrated on:

- Adding a new syscall and user tool to observe process state.
- Extending `struct proc` with fields required by MLFQ.
- Defining queue structures and helper functions for enqueue and dequeue.
- Updating the build system to include new user-space utilities.

3.2 New Syscall: `getprocinfo`

A new syscall, `getprocinfo(pid, struct procinfo *buf)`, was added to gather a snapshot of process state into a user-provided structure. The kernel populates a `procinfo` record with fields such as:

- PID and current state,
- base priority and current queue level,
- remaining time slice,
- total runtime and per-queue runtime counters.

This information is then copied to user space, allowing user programs to inspect scheduling-related details in a controlled way.¹

3.3 User Tool: `procinfo`

A companion user program, `user/procinfo.c`, uses the `syscall` to print a textual summary of a process. This includes its basic metadata and the MLFQ fields described above. The tool is useful for confirming that all bookkeeping fields are initialized correctly and evolve as expected during execution.

3.4 Process Bookkeeping Fields

To support MLFQ, the following fields were added to `struct proc`:

- `base_priority`: initial priority for the process.
- `queue_level`: current MLFQ level (0 as highest).
- `time_slice_budget`: remaining ticks in the current time slice.
- `total_runtime`: total ticks spent running.
- `queue_runtime[MLFQ_LEVELS]`: per-level runtime counters.
- `mlfq_next`: pointer for queue linkage in a per-level FIFO list.

These fields are initialized in a helper `mlfq_reset_proc()` function, which sets the default level and time slice, and clears runtime counters.

3.5 MLFQ Parameters and Queue Structure

The system uses four levels, with level 0 as the highest priority. Time quanta double with each lower level:

Level	Priority	Time Quantum (ticks)
0	highest	3
1	high	6
2	medium	12
3	low	24

A global MLFQ state holds an array of per-level queues. Each queue is a simple FIFO list implemented using `mlfq_next` pointers and a `head` and `tail`. Initialization code sets all pointers to `NULL`, providing a clean starting state.

4 Week 2: Core Scheduler Implementation

4.1 Queue Structures and Locking

The MLFQ state is guarded by a spinlock to allow safe concurrent access from multiple CPUs. Each level holds a `procqueue` with `head` and `tail` pointers, and operations such as `mlfq_enqueue` and `mlfq_dequeue` preserve FIFO order.²

4.2 Enqueue and Dequeue Policy

A helper `mlfq_make_runnable()` is responsible for:

- Checking that a process is actually in the `RUNNABLE` state.

¹See the Week 1 design document for details on the `syscall` interface and fields recorded in the `procinfo` structure.

²The Week 2 document gives C snippets for the queue state and enqueue/dequeue helpers.

- Optionally resetting its time-slice budget.
- Inserting it into the chosen queue at either the head or tail, depending on the context (e.g., wakeup vs. yield).

This centralizes the logic for placing a process onto an MLFQ queue.

4.3 Scheduler Tick Accounting

Timer interrupts invoke `scheduler_tick()` on the current CPU. This function:

- Confirms the current process is valid and running.
- Increments `total_runtime` and the runtime counter for the current `queue_level`.
- Decrements the `time_slice_budget`, if positive.
- When the budget reaches zero, demotes the process to the next lower level (if not already at the lowest level), resets its budget, and returns a flag indicating that a reschedule is needed.

This separates time accounting from the actual scheduling decision, which simplifies the main scheduler loop.

4.4 Scheduler Loop

The main `scheduler()` function repeatedly:

1. Enables interrupts, then disables them before picking a process.
2. Calls `mlfq_pick_next()` to select the next runnable process from the highest non-empty queue.
3. If no process is runnable, executes a low-power instruction and repeats.
4. When a process is found, sets its state to `RUNNING`, switches to its context, and then returns to the scheduler when the process yields, sleeps, or exits.
5. If the process is still `RUNNABLE` after the context switch, it is placed back on the appropriate queue via `mlfq_make_runnable()`.

4.5 Queue and Wakeup Policy

To favor interactive workloads:

- Newly created processes start in the highest-priority queue with a full time slice.
- Processes that sleep and then wake are inserted near the front of the appropriate queue, so I/O-bound tasks see lower latency.
- Processes that voluntarily yield keep their current level but receive a refreshed budget.

4.6 Demo Program

A user program, `testmlfq`, spawns both a CPU-bound worker and an I/O-bound worker. The parent periodically samples their statistics using `getprocinfo` and prints the queue level, remaining budget, and runtime for each process. The output shows the CPU-bound task gradually drifting to lower queues, while the I/O-bound task tends to remain in a higher-priority queue.

5 Week 3: Priority Boosting and Polishing

5.1 Global Priority Boosting

Week 3 focused on preventing starvation of long-running tasks by introducing a global priority boost. A constant interval, such as `MLFQ_BOOST_INTERVAL = 100` ticks, defines how often a full boost occurs.³

³The Week 3 report gives the exact constant and code skeleton for the boost function.

At each timer tick, `scheduler_tick()` invokes `mlfq_maybe_boost()` on CPU 0. When the number of ticks since the last boost exceeds the interval, a boost is performed.

5.2 Boost Procedure

The boost procedure:

- Updates a global `last_boost_tick` counter.
- Clears all per-level queues.
- Resets every non-UNUSED process to the top-level queue with a fresh time slice.
- Re-queues any RUNNABLE process in the highest-priority queue.

Running processes inherit the new level the next time they yield or block. This approach keeps the boost logic simple and keeps the scheduler state consistent.

5.3 Testing Syscall: `boostproc()`

To test boosting deterministically, a new syscall `boostproc()` was exposed. Its implementation is small: it calls the internal `mlfq_force_boost()` helper and returns to user space.

The demo program `testmlfq` was extended to accept an optional argument indicating at which iteration a manual boost should be triggered. When that iteration is reached, it prints a message and calls `boostproc()`.

5.4 Scheduler Integration and Behavior

With boosting in place, the timer tick now has three main responsibilities:

- Charge CPU time to the currently running process.
- Handle time-slice expiration and demotion to lower levels.
- Periodically trigger MLFQ priority boosting on CPU 0.

This combination keeps CPU-bound tasks from dominating low-level queues forever and gives long-running jobs a chance to run at higher priority again after each boost cycle.

5.5 Observations from Demo Runs

Empirical observations from `testmlfq` runs include:

- The CPU-bound process descends from level 0 to lower levels as it repeatedly uses full time slices.
- The I/O-bound process tends to remain at the top level, since it yields frequently and often re-enters the runnable state via wakeups.
- After a manual or automatic boost, all processes are reset to level 0, and scheduling resumes with a clean high-priority queue.

These results match the intended behavior of the MLFQ design and confirm that both demotion and boosting work as expected.

6 Testing and Evaluation

The main evaluation method was the `testmlfq` program, combined with the `procinfo` tool. Together they allowed us to:

- Monitor queue levels, remaining budget, and runtime per process.
- Compare behavior of CPU-bound versus I/O-bound tasks.
- Confirm that interactive tasks receive prompt service.
- Verify that long-running tasks eventually receive a global priority refresh via boosting.

Further testing could include:

- Multiple CPU-bound processes competing at lower levels.

- Mixed workloads that combine compute-intensive phases and I/O phases.
- Stress tests with many short-lived tasks entering and leaving the system.

7 Conclusion and Future Work

We implemented a full MLFQ scheduler for `xv6`-RISC-V, including:

- Per-process MLFQ metadata and observability through `getprocinfo`.
- A queue-based scheduler that selects processes from multiple priority levels with different time quanta.
- Time-slice tracking, demotion logic, and policies that favor I/O-bound processes.
- A global priority boost mechanism that prevents starvation, exposed both internally and via a user-visible `boostproc()` syscall.

Possible extensions include experimenting with different time quantum configurations, adapting the boost interval based on load, or integrating additional metrics such as recent I/O activity to refine promotion policies.